

CONSIDERACIONES GENERALES PARA RESOLVER LOS EJERCICIOS

- No podrá ser corregido ningún programa que no compile.
- No agregar ni quitar funcionalidades.
- No modificar los formatos.
- No utilizar `getch` ya que Linux no lo acepta.
- No utilizar `strcpy`, se deberá crear una función o procedimiento propio.
- Si el examen es escrito debe ser entregado en tinta, con cada hoja correctamente numerada y en el margen superior de cada hoja se debe indicar a qué archivo corresponde dicho código.
- La nota dependerá de la eficiencia.

ÍNDICE

Etapas 1 — Bucles anidados (dibujos)

1) Fila	2
2) Cuadrado	2
3) Triángulo creciente	2
4) Triángulo decreciente	2
5) Cuadrado hueco	2
6) Pirámide	3
7) Pirámide invertida	3
8) Columna con guiones	3
9) Escalera con guiones	3
10) Zigzag hacia la izquierda	4
11) Zigzag hacia la derecha	4
12) Diagonal a la derecha con saltos	4
13) Diagonal a la izquierda con saltos	5

Etapas 2 — Arrays y matrices con informes

14) Jugadores y goles (3 × 3)	5
15) Estudiantes y notas (3 × 3)	5
16) Sucursales y ventas (3 × 3)	6
17) Vector: pares e impares	6
18) Matriz de kioscos + reporte.txt	6
19) Matriz de kioscos	6
20) Matriz de ganancias por empresa	7
21) Matriz de equipos y goles (5 × 5)	7

Etapas 3 — TDA básicos

22) TDA Círculo	8
23) TDA Negocio de peatonal	8
24) TDA Fracción	8

25) TDA Factura de energía eléctrica	8
--	---

Etapas 4 — TDA con arrays y estructuras anidadas

26) TDA Empleo con empleados	9
27) Estudiante y materias	9
28) Facturas METROGAS	10
29) Usuarios y años bisiestos	11

Etapas 5 — TDA con aleatorios, ordenamiento y archivos

30) Padrón electoral	11
31) ABM de vehículos y pasajeros	11
32) Persona, Fecha y Vehículo	11

Etapas 6 — Listas, pilas, colas, ordenamiento y búsqueda

33) Lista con todas las primitivas	12
34) Pila, Cola y Lista (teoría + código)	12
35) Lista, Pila, Cola y burbuja (teoría)	12
36) Ordenamiento por inserción	12
37) Búsqueda binaria por DNI	13
38) Búsqueda secuencial por DNI	13

Etapas 7 — Programas completos con menú

39) Personas y sueldos en archivo	13
40) DNI y nombres ordenados en archivo	14
41) Jugadores ordenados por goles	14
42) Lluvia por ciudad y semana	14
43) Ventas por local y producto	15
44) Factorial, primo y recursividad	16

- 1) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

- 2) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

- 3) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

*
**

- 4) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

**
*

- 5) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

* *
* *
* *

- 6) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

```

      *
     ***
    *****
   ********

```

- 7) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

```

   *****
  *****
 *****
  ***
   *

```

- 8) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

```

-----*
-----*
-----*
-----*
-----*

```

- 9) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones **for**.

```

-----*
----*
--*
-*
*

```

- 10) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones `for`.

```

      *
     *
    *
   *
  *
 *
*

```

- 11) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones `for`.

```

*
 *
  *
   *
    *
   *
  *
 *
*

```

- 12) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones `for`.

```

*

*

*

*

*

```

- 13) Hacer un programa que permita hacer el siguiente dibujo imprimiendo de a un carácter a la vez y usando combinaciones `for`.

```

      *
    *
  *
*
*
  
```

- 14) Hacer un programa que permita cargar los nombres de tres jugadores de fútbol y los goles que hizo cada uno en 3 partidos jugados en el mes.

Luego deberá mostrar un informe como el que se detalla a continuación.

Jugador	Partido 1	Partido 2	Partido 3	Promedio
A	0	2	3	1.66
B	2	1	0	1
C	1	3	1	1.66
Jugador que hizo más goles	Jugador B	Jugador C	Jugador A	

(valores de ejemplo)

Indicar además quién no hizo goles en cada partido.

- 15) Hacer un programa que permita cargar los nombres de tres estudiantes y las notas de cada uno para tres materias.

Luego deberá mostrar un informe como el que se detalla a continuación.

Estudiante	Materia 1	Materia 2	Materia 3	Promedio
Juan	4	2	10	5,33
Pedro	7	8	9	8,00
María	4	10	8	7,33
Nota más alta	7	10	10	

(valores de ejemplo)

- 16) Hacer un programa que permita cargar los nombres de tres sucursales de una empresa y las ventas de cada uno para tres productos.

Luego deberá mostrar un informe como el que se detalla a continuación.

Sucursal	Producto 1	Producto 2	Producto 3	Promedio
Lanús	4	2	10	5,33
Avellaneda	7	8	9	8,00
Banfield	4	10	8	7,33
Local que menos se vendió	Avellaneda	Lanús	Banfield	

(valores de ejemplo)

- 17) Realizar un programa que permita cargar un vector de 8 números y diga cuántos son pares y cuantos impares.

- 18) Se tiene un total de 10 kioscos, donde se han guardado el total de cigarrillos vendidos mes por mes. Se pide cargar una matriz para almacenar todos estos datos, cargar por teclado. Luego mostrar dicha matriz. Una vez llena la matriz, se pide calcular el promedio de cigarrillos vendidos por cada kiosco en el año y el promedio de cigarrillos por mes, para todos los kioscos. Estos promedios deben ser guardados en vectores. Todos los promedios deben ser mostrados por pantalla. Las funciones y procedimientos necesarias son las dadas a continuación (deben ser respetadas):

- `void cargarMatriz(int matriz[10][12]);`
- `void mostrarMatriz(int matriz[10][12]);`
- `void calcularPromKioscos(int matriz[10][12], float promKioscos[10]);`
- `void calcularPromMes(int matriz[10][12], float promMes[12]);`
- `void mostrarPromKioscos(float promKioscos[10]);`
- `void mostrarPromMes(float promMes[12]);`
- `int maximoMatriz(int matriz[10][12]);`
- `void generarReporte(int matriz[10][12]);`

Se pide identificar la máxima cantidad de cigarrillos vendidos. Todo debe ser llamado desde el `main` en el orden correcto las veces que sea necesario y creando los arreglos que sean necesarios de forma local en el `main`. Por último, se debe generar un reporte y guardarlo en un archivo `reporte.txt`. Ese reporte tiene que generarse al utilizar el procedimiento `generarReporte`. En el mismo se debe ver la matriz, el máximo de la matriz, y los vectores promedios.

- 19) Se tiene un total de 10 kioscos, donde se han guardado el total de cigarrillos vendidos mes por mes. Se pide cargar una matriz para almacenar todos estos datos, cargar por teclado. Luego mostrar dicha matriz. Una vez llena la matriz, se pide calcular el promedio de cigarrillos vendidos por cada kiosco en el año, y el promedio de cigarrillos por mes para todos los kioscos. Estos promedios deben ser guardados en vectores. Todos los promedios deben ser mostrados por pantalla.

Las funciones y procedimientos necesarios son las dadas a continuación, deben ser respetadas. Por último, se pide identificar la máxima cantidad de cigarrillos vendidos. Todo debe ser llamado desde el `main` en el orden correcto las

veces que sea necesario y creando los arreglos que sean necesarios de forma local en el `main`. Las funciones para utilizar son las siguientes:

- `void cargarMatriz(int matriz[10][12]);`
- `void mostrarMatriz(int matriz[10][12]);`
- `void calcularPromKioscos(int matriz[10][12], float promKioscos[10]);`
- `void calcularPromMes(int matriz[10][12], float promMes[12]);`
- `void mostrarPromKioscos(float promKioscos[10]);`
- `void mostrarPromMes(float promMes[12]);`
- `int maximoMatriz(int matriz[10][12]);`

20) Crear una matriz de 12 columnas y 4 renglones, de números decimales, cada valor explica las ganancias de 4 empresas distintas mes a mes. Cargar la matriz por teclado.

- A) Encontrar el total ganado por cada empresa al año.
- B) Encontrar cuanto se ganó por mes contando a todas las empresas juntas.
- C) Calcular el total ganado por todas las empresas juntas en el año.
- D) Encontrar que empresa gano más y en qué mes.
- E) Que empresa gano menos y cuándo.
- F) Calcular el promedio de ganancias de cada empresa por bimestres. ¿Cuál gano más en cada bimestre y en qué bimestre?

21) Hacer un programa que permita cargar una matriz en la que figuren 5 equipos de futbol y los goles realizados por cada uno en 5 partidos. Si para un equipo y un partido, se cargan más datos de una vez, solo se tendrá en cuenta la última carga, que deberá pisar los datos anteriores.

Cargados los datos, deberá mostrar el total de goles por equipo y el total de goles por partido, tal como se muestra en el ejemplo.

Nro.	Equipo	Partido 1	Partido 2	Partido 3	Partido 4	Partido 5	Goles por equipo
1	aaa	1	0	0	3	2	6
2	bbb	0	3	0	2	1	6
3	ccc	0	0	0	1	1	2
4	ddd	0	0	0	0	0	0
5	eee	4	2	2	0	3	11
	Cantidad de goles	5	5	2	6	7	25

(valores de ejemplo)

Luego se mostrará un resumen de goles por equipo, ordenado de más goles a menos goles, y por último, la cantidad promedio de goles que se hicieron.

- 22) Crear un TDA para describir los datos de un **Círculo**. La descripción total de un círculo viene dada por su punto central, en el eje X y el punto del eje Y, además del radio. Crear las funciones en el archivo `.h`, `.c` y probarlas en el `main`.

La estructura para respetar debe ser la siguiente:

```
Struct Circulo{
    float puntoX;
    float puntoY;
    float radio;
    float perimetro;    // PI por diámetro
    float area;         // PI por radio al cuadrado
}
```

El programa debe tener:

- A) `mostrarCirculo (Circulo c);`
- B) `crearCirculo(float x, float y, float radio);` (los otros parámetros se calculan aquí dentro)
- C) Los gets y sets necesarios.

- 23) Crear un TDA, completo, donde se guarden y utilicen los datos de “negocios” de una peatonal. Los datos a usar son:

- `char nombreNegocio[100]`
- `int altura`
- `int piso`
- `char vereda` (p o i, par o impar)

Crear los archivos `.h`, `.c` y el `main` con todas las primitivas realizadas e implementadas en el `main`.

La primitiva crear, constructor debe tener esta forma:

```
void crearNegocio(char negocio[100], int altura, int piso)
```

- 24) Crear un TDA completo para describir el funcionamiento de las fracciones. La estructura debe tener un numerador, denominador (ambos enteros) y un `valorEnDecimalDeFraccion float`.

Dentro del TDA, además de las funciones y procedimientos generales, se piden las siguientes funciones:

- `Fracción calcularSuma (Fracción f1, Fracción f2);`
- `void relacionar (Fracción f1, Fracción f2);` // f1 es mayor/menor que f2
- `Fraccion simplificarFraccion (Fracción f1);` // $2/8 \rightarrow \frac{1}{4}$

Todo debe ser llamado de la forma correcta en el `main`.

- 25) Realizar un TDA completo que permita administrar las facturas de una empresa de energía eléctrica. En la estructura debe tener un `nombreEmpresa`, `nroFactura`, `consumo (KWh)`, `precioPorKWh`, `ultimoConsumo (KWh)`, `precioTotal`, `tipoDeConsumidor` (debe ser un enum, 0- Común, 1-Con Descuento).

Si se consumió menos que el consumo anterior el precio total debe tener un descuento del 10%, en el caso que se tenga descuento la tarifa pasa a ser la mitad (pueden sumarse ambos beneficios). Todo debe ser ejecutado desde el `main`.

26) Crear un TDA completo para administrar un **Empleo**, con un nombre, teléfono, dirección, y con un máximo de 100 empleados. El TDA empleado no hay que realizarlo, pensar que ya está creado. Desde el `main` se tiene que crear un menú con enumeradores, que permita, crear **Empleo**, mostrar Empleo (y sus empleados), agregar **Empleado**.

27) Se pretende crear un programa para administrar las materias de un determinado estudiante. Para esto tendremos un **Estudiante** con un array de **Materia** (10 elementos). Del estudiante solo nos interesa saber su nombre y su edad. Mientras que de las materias queremos saber su carga horaria semanal, el profesor titular y la cantidad de inscriptos.

Una vez creado el esquema correspondiente se pide:

- 1- Crear por parámetro 4 materias. Crear un estudiante. Agregar las 4 materias a este estudiante. Mostrar el estudiante.
- 2- Ordenar las materias del estudiante según el titular de las materias. Mostrar al estudiante.
- 3- Calcular e imprimir por pantalla la cantidad de inscriptos a las materias de este estudiante.
- 4- Eliminar una materia (m3). Mostrar al estudiante.

En el `main` deberán escribir esto y solo esto, sin modificar nada, solo sus datos personales:

```
int main()
{
    printf("NOMBRE APELLIDO - DNI: 12345678 \n");

    printf("\n\nEJERCICIO 1 - Crear y agregar materias a estudiante");
    Materia m1 = crearMateria(4, "Jorge G", 90);
    Materia m2 = crearMateria(8, "Federico R", 56);
    Materia m3 = crearMateria(6, "Roberto G", 105);
    Materia m4 = crearMateria(4, "Edgardo D", 75);

    Estudiante e1 = crearEstudiante("Estudiante 1", 23);
    agregarMateria(e1, m1);
    agregarMateria(e1, m2);
    agregarMateria(e1, m3);
    agregarMateria(e1, m4);
    mostrarEstudiante(e1);

    printf("\n\nEJERCICIO 3 - Mostrar cantidad de alumnos");
    printf("\nLa cantidad de alumnos que cursan junto al estudiante: %d", cantidadDeAlumnos(e1));

    printf("\n\nEJERCICIO 4 - Eliminar materia");
    eliminarMateria(e1, m3);
    mostrarEstudiante(e1);

    return 0;
}
```

28) Desarrollar un programa que permita facturar los consumos de determinados usuarios en distintos periodos para la empresa METROGAS. De la factura solamente nos interesa saber quién es el “Usuario”, cuál es la “Fecha”, cuál es el “Consumo” (sería en metros cúbicos, m³). Con el consumo se debe calcular el “Total” de la factura.

Para calcular el total, hay que sumarle al costo fijo de \$200 el producto del consumo con el coeficiente del precio. El coeficiente es 15.5 si se gastaron menos de 100m³, es 18.9 si se gastaron entre 100 y 300 m³ y es de 21.9 si se gastaron más de 300 m³.

Por ejemplo: Si se consumieron 130, el total es $130 \times 18.9 + 200$.

Del usuario nos interesa conocer su “Nombre y apellido” y su “Número de cuenta”. De la fecha queremos conocer su “mes” y su “año”.

Programa:

- 1- Crear tres facturas, con sus usuarios y fechas.
- 2- Crear un array de facturas, con esas 3 facturas del ítem 1.
- 3- Mostrar las facturas.
- 4- Ordenar las facturas por nombre del cliente.
- 5- Mostrar las facturas ordenadas.

El main debe verse así, sin excepciones:

```
int main()
{
    printf("----- FINAL MAYO -----\n");
    printf("ESTUDIANTE: daianita ----- DNI 12345678\n");

    // Creamos 3 facturas
    Usuario u1 = crearUsuario(1111, "Brenda Perez");
    Fecha fecha1 = crearFecha(3, 2021);
    Factura f1 = crearFactura(120, u1, fecha1);

    Usuario u2 = crearUsuario(2222, "Jorge Golfieri");
    Fecha fecha2 = crearFecha(1, 2021);
    Factura f2 = crearFactura(411, u2, fecha2);

    Usuario u3 = crearUsuario(3333, "Alejandra Vranic");
    Fecha fecha3 = crearFecha(5, 2020);
    Factura f3 = crearFactura(33, u3, fecha3);

    Factura facturas[3] = {f1, f2, f3}; // Creamos un array con las 3 facturas
    mostrarFacturas(facturas, 3);      // Las mostramos
    ordenarFacturas(facturas, 3);      // Las ordenamos
    mostrarFacturas(facturas, 3);      // Las volvemos a mostrar

    return 0;
}
```

29) Utilizando una estructura llamada **Usuario**, crear las funciones y procedimientos necesarios para cargar 10 usuarios, mostrar los diez usuarios. El usuario debe tener **dni**, **nombre**, **contraseña** y **fecha de nacimiento**, la cual es una estructura formada por tres enteros (**día**, **mes**, **año**). También se pide mostrar los usuarios ordenados por año de nacimiento. Mostrar por pantalla la cantidad de usuarios que han nacido un año bisiesto. Todo debe ser invocado desde el **main**.

30) Crear un programa que permita administrar al padrón electoral. Para esto crear un array de, como mucho, 10 ciudadanos. De los ciudadanos nos interesa saber su nombre, apellido y su dni (debe ser aleatorio entre 10.000 y 100.000). A su vez el ciudadano tiene una localidad, de la cual nos interesa saber su nombre y su código postal.

Se pide:

- 1- Crear a estas 5 personas, agregarlas al array y mostrar los ciudadanos cargados (constructor por parámetros, no por teclado):
 - Nicolas Pérez, de Lanús, 1824
 - Jorge Gofieri, de Banfield, 1825
 - Alejandra Vranic, de Lanús, 1824
 - Leandro Ríos, de Lomas, 1832
 - Roberto García, de Banfield, 1825
- 2- Ordenarlas por CP, y mostrar a los ciudadanos cargados ordenados.
- 3- Eliminar a los ciudadanos de Banfield.
- 4- Guardar el array en uno o varios archivos **.txt** con algún formato de fácil recuperación.

31) Construir un programa para realizar los ABMs de vehículos y sus pasajeros. De los vehículos nos interesa conocer su patente, su capacidad (cantidad de pasajeros) y marca, la patente debe ser aleatoria y alfanumérica. De los pasajeros queremos saber su nombre, apellido y su dni. Este último debe ser aleatorio entre 6 y 8 cifras.

Con el programa se pretende que:

- 1- Se genere un vehículo de 5 pasajeros, y se muestre la información (sin pasajeros).
- 2- Se agregue un pasajero al vehículo, y se muestre la información.
- 3- Que se agreguen (trate de agregar) 6 pasajeros más y se muestre la información.
- 4- Que se elimine un pasajero, y se muestre la información (pasajero buscado por dni).
- 5- Que se ordenen los pasajeros por nombre y se muestre la información.
- 6- Que se guarde la información en un **.txt**

32) Se tienen 3 estructuras anidadas: **Persona**, **Fecha** y **Vehículo**.

Las personas tienen una fecha de nacimiento (año, mes, día), UN SOLO vehículo (**int modelo**, **char[10] patente**), nombre, apellido y DNI.

Crear un menú con enumeradores, que permita:

- 1) Decidir cuantas personas cargar
- 2) Cargar las personas
- 3) Ver las personas
- 4) Ordenar las personas por patente de vehículo

No pueden faltar las siguientes funciones/procedimientos:

- `void ordenarPorPatente(Struct Persona personas[], int cantidad);`
- `int busquedaBinariaDNI (Struct Persona personas[], int cantidad, int busco);`
- `void guardarEnArchivo(Struct Persona personas[], int cantidad);` (con cualquier formato se debe guardar, pero delimitando con algún carácter)

Todo debe ser llamado de la forma correcta en el `main`.

33) Utilizando en el `main` TODAS las funciones y procedimientos vistos para la manipulación de Listas, crear una lista que termine con la siguiente colección de números: `3,-3,1,6,7,8,11,-2`. Las funciones eran las siguientes (usarlas todas mandando los parámetros necesarios en cada una):

- | | |
|-------------------------------|--------------------------------|
| • <code>crearLista</code> | • <code>remove</code> |
| • <code>insertarInicio</code> | • <code>obtenerElemento</code> |
| • <code>insertarFin</code> | • <code>obtenerTamano</code> |
| • <code>insertar,</code> | • <code>imprimir</code> |
| • <code>removeInicio</code> | • <code>esVacia</code> |
| • <code>removeFin</code> | • <code>liberarLista</code> |

34) ¿Cuáles son las funciones que caracterizan a una estructura de datos de Pila y Cola? ¿En qué se diferencian de la lista? Utilizar TODAS las funciones de las listas desde el `main` para enlistar números enteros. Al finalizar de ese algoritmo la lista debe quedar: `2-3-4-8-11-7-10`.

35)

- A) Explique las diferencias entre una Lista, una Pila y una Cola. ¿Cómo haría para generar la siguiente colección de números con estas 3 estructuras de datos: `1,3,-1,-9,5,11`? Sin hacer el código fuente, solo explicar cómo es el algoritmo.
- B) Supongamos que se tiene un array de `int`, `1,3,-1,-9,5,11`, ¿Cómo funciona el método burbuja? Realizar una prueba de escritorio.

36) Explicar con un esquema o prueba de escritorio, como sería el ordenamiento de inserción para el siguiente vector entero: `(-3,4,1,-2,5,6)`. Una vez explicado crear el procedimiento para ordenar cualquier vector de números enteros.

37) Realizar un programa que haga lo siguiente:

Menú:

- 1) Cargar datos
 - 2) Buscar datos
 - 3) Imprimir datos
 - 0) Finalizar programa

La carga de datos permitirá ingresar información de 5 personas. Los datos a ingresar son:

- dni (numérico)
- nombre (char(20))

El nombre se deberá ingresar con `scanf` por lo cual no deberá tener espacios.

Ingresando un DNI, debe realizar una búsqueda binaria y si existe, mostrar el nombre, si no existe lo debe informar y volver al menú.

Impresión deberá mostrar los DNIs y nombres en el orden en el que se encuentran.

38) Realizar el mismo ejercicio anterior, pero ingresando un DNI, debe realizar una búsqueda secuencial en vez de binaria, y si existe, mostrar el nombre. Si no existe lo debe informar y volver al menú.

39) Realizar un programa que haga lo siguiente:

Menú:

- 1) Borrar datos
 - 2) Cargar datos
 - 3) Ordenar por nombre
 - 4) Ordenar por DNI
 - 5) Mostrar datos
 - 0) Finalizar programa

Por medio de este menú se generará un archivo de personas que contendrá los siguientes datos:

- dni (entero)
- nombre (sin espacios)
- sueldo (entero)

Cargar datos: permitirá cargar datos hasta que `dni = 0`

Mostrar datos: mostrará la información cargada en el último orden seleccionado, totalizando al final la suma de los sueldos.

El programa deberá permitir que los datos se conserven si se cierra y se vuelve a abrir, es decir que los debe guardar en un archivo (de cualquier tipo).

40) Realizar un programa que permita cargar en memoria los siguientes datos:

- nroDni (numérico)
- nombre (20 caracteres)

Los datos se cargarán en cualquier orden y en cualquier cantidad (no mayor a 20).

Al ingresar `nroDni = 0` el programa deberá guardar en un archivo los datos ordenados alfabéticamente por nombre.

Cada vez que se inicia la carga de datos (no el programa) deben borrarse los datos cargados anteriormente.

Menú:

- 1) Vaciar archivo
- 2) Cargar datos
- 3) Mostrar datos ordenados por nombre
- 4) Mostrar datos ordenados por DNI
- 0) Finalizar programa

41) Realizar un programa que haga lo siguiente:

Menú:

- 1) Cargar datos
- 2) Imprimir
- 0) Finalizar programa

La carga de datos permitirá ingresar información de 5 jugadores de fútbol. Los datos a ingresar son:

- dni (numérico)
- nombre (char(20))
- cantGoles (entero)

El nombre se deberá ingresar con `scanf` por lo cual no deberá tener espacios.

La opción de impresión debe mostrar la lista de jugadores con la cantidad de goles, de mayor a menor.

42) Realizar un programa que haga lo siguiente:

Menú:

- 1) Borrar datos
- 2) Cargar datos
- 3) Ordenar por ciudad
- 4) Ordenar por semana
- 5) Mostrar datos
- 6) Mostrar resumen
- 7) Mostrar estadísticas
- 0) Finalizar programa

Los datos a cargar en memoria, vectores, matrices, estructuras, etc:

- ciudad (hasta 5 caracteres)
- semana (valores 1, 2, 3 o 4)
- mmLluvia (milímetros de lluvia caída, numérico)

Se podrá cargar un máximo de 30 líneas. La ciudad y la semana podrán repetirse, por lo cual se deberá acumular con los valores existentes para esa ciudad y esa semana.

Resumen:

Ciudad	Semana 1	Semana 2	Semana 3	Semana 4	Total
A	10	0	9	6	25
B	15	7	5	7	35
C	0	18	7	1	26
Total	25	25	21	14	85

(valores de ejemplo)

Estadísticas:

Ciudad	Promedio	Máx	Min <>0	=0
A	6,25	10	6	2
B	8,5	15	5	
C	6,5	18	1	1
Total	21,25	25	14	

(valores de ejemplo)

43) Hacer un programa que permita por medio de un menú hacer las siguientes tareas:

- 1) Vaciar el archivo
- 2) Cargar datos
- 3) Mostrar cómo fueron cargados
- 4) Mostrarlos ordenados por local
- 5) Mostrarlos ordenados por producto
- 6) Mostrar cuadro
- 0) Salir

Cargar ventas de locales (hasta 4) de distintos productos (hasta 4), se ingresará además la cantidad vendida (entero).

Se deberán seguir cargando datos hasta que el número de local sea igual a 0.

Los datos deberán ser grabados en un archivo y deberá ser posible salir y volver a entrar en el programa sin perder los datos.

El archivo puede ser de cualquier tipo.

44) Crear un menú, con las siguientes opciones:

- A) Ingresar un numero entero
- B) Calcular su factorial
- C) Averiguar si es número primo
- D) Mostrar todos los números del cero al número ingresado, o del número ingresado al cero, según el signo, utilizando recursividad.

Todas las opciones deben estar realizadas utilizando funciones o procedimientos.